

„One Pager“ on how to create Panel ID for Politicians from different Countries

Steps Overview:

1. Code Panel ID in Python via VS Code
2. Test Performance in R
3. Download final Excel

1. Code Panel ID in Visual Studio Code (VSCode)

⇒ **Link:** <https://github.com/oettlquin/Politikernamen-Panel-ID/blob/main/General%20Code.py>

1.1. Preparation:

- **Download VSCode:** <https://code.visualstudio.com>
- **Use Python version 3.10.12**
 - Ensures compatibility between *linktransformer* and *torch* libraries.
- **Install Extensions**
 - Open (on Mac) with Cmd + Shift + X.
 - Search and install “Python”.
 - Search and install “Jupyter”.
- **Install in Terminal:**

```
pip install torch==2.3.0
pip install linktransformer==0.1.15
pip install linktransformer numpy pandas torch
pip install --upgrade torch transformers linktransformer
```

- It is crucial to put this in the terminal.
- These install and update the required libraries for the code to work.
- *torch* is a machine learning library required by *linktransformer*. Version 2.3.0 ensures compatibility.
- *linktransformer* loads the pretrained large language model. Version 0.1.15 for stability.

- *numpy* and *pandas* are standard Python libraries for data manipulation and numerical operations.
- *transformers* is included in the upgrade step to ensure the latest features and compatibility with *torch* and *linktransformer*.
- Using *pip install --upgrade* ensures that the libraries (*torch*, *transformers*, and *linktransformer*) are updated to the latest stable versions compatible with each other.

1.2. Explanation of the whole Code

1.2.1. # Imports

```
import numpy as np
import pandas as pd
import os
from sklearn.metrics.pairwise import cosine_similarity
from scipy.spatial.distance import squareform
from scipy.cluster.hierarchy import fcluster, linkage
import linktransformer as lt
```

- *os* is used for file path management.
- *sklearn.metrics.pairwise.cosine_similarity* calculates pairwise cosine similarities.
- *scipy.spatial.distance.squareform* and *scipy.cluster.hierarchy.linkage* handle distance matrix conversion and hierarchical clustering.
- *linktransformer* as *lt* is a way to give the *linktransformer* module a shorter and more convenient alias (*lt*) in the code.

1.2.2. # Load the Model

```
try:
    model = lt.load_model("sentence-transformers/multi-qa-MiniLM-L6-cos-v1")
    print("Model loaded successfully!")
except Exception as e:
    print(f"Error loading model: {e}")
    exit()
```

- loads the pretrained model optimized for semantic similarity tasks. It maps text (in our case, politician names) to a high-dimensional vector space where semantically similar names are close together.
- If the model fails to load, an error message is displayed, and the script exits.

1.2.3. # Load the Dataset

```
# Path to your .dta file
local_file_path = "/your_file_path/Chile_master_1989-2017.dta"

# Try to load the dataset
try:
    country_df = pd.read_stata(local_file_path)
    print("Dataset loaded successfully!")
except Exception as e:
    print(f"Error loading dataset: {e}")
    exit()

# Display first few rows to ensure it loaded correctly
print(country_df.head())
```

- The dataset is successfully loaded and contains a column named “candidate,” which includes the names of the politicians.
- If the file cannot be read, the script will print an error message and exit.
- Replace the placeholder with your file path and the name of your dataset.
- The first five rows of the dataset are displayed to verify that the data has been loaded correctly.

1.2.4. # Generate Semantic Embeddings

```
def generate_embeddings(df, batch_size=500):
    embeddings = []
    for start in range(0, len(df), batch_size):
        end = min(start + batch_size, len(df))
        batch = df["candidate"].iloc[start:end].fillna("").tolist()
        batch_embeddings = model.encode(batch)
        embeddings.extend(batch_embeddings)
    return np.array(embeddings)
```

```
embeddings = generate_embeddings(country_df)
```

- **Purpose:**

- The function *generate_embeddings* converts each name in the "candidate" column into a high-dimensional vector (here 384 dimensions). These embeddings capture the semantic meaning of the names, helping the model recognize similar names despite typos or variations.
- Important: the politician names ("candidate") from different batches are also compared, the purpose of batching is simply to process the data in manageable chunks to avoid running out of memory or computational capacity (see the following bullet point). The embeddings for all politician names - regardless of which batch they came from - are combined into one large array after processing. This ensures that all of them are included in the following similarity calculations.

- **Batch Processing:**

- Since encoding large datasets at once can be memory-intensive, politician names are processed in batches of *batch_size=500*. Mostly needed for datasets with a size of >15,000 observations.
- For each batch, the *model.encode* function generates embeddings, which are then added sequentially into a single array.

1.2.5. **# Compute the Full Cosine Similarity Matrix**

```
print("Computing cosine similarity matrix...")  
similarity_matrix = cosine_similarity(embeddings)
```

- *cosine_similarity(embeddings)* computes a similarity score between all pairs of embeddings. The resulting matrix has values between -1 and 1, where 1 indicates identical vectors and values closer to 0 (at 0, vectors are orthogonal) or negative indicate dissimilarity (at -1, vectors are completely opposite).

1.2.6. **# Convert Similarity Matrix to Distance (1 - Similarity)**

```
distance_matrix = 1 - similarity_matrix
```

- Because we do hierarchical clustering, a distance matrix is required, as it operates based on distances. It measures how “far apart” politician names are to decide how to merge or split clusters. Using cosine similarity directly would contradict this logic since the algorithm expects smaller values to be closer relationships.
- To transform similarity into a distance matrix, the formula $1 - \text{similarity}$ is applied. This ensures that larger distances correspond to more dissimilar pairs.
 - 0 : Perfectly similar
 - 1 : Completely dissimilar
 - This formula is the reason why a decrease in the threshold makes the model stricter
- Overall, while cosine similarity measures how close names are, which works well for evaluating individual pairwise relationships, distance is required for clustering because it aligns with the algorithm’s expectations and logic. And it allows for applying thresholds in a way consistent with the hierarchical structure.

1.2.7. # Clamp Negative Values to Zero

```
distance_matrix = np.maximum(distance_matrix, 0)
```

- Occasionally, some values might be slightly negative. These are restricted to zero using `np.maximum(distance_matrix, 0)`, mainly for practical reasons (keeping fixed range from 0 to 1). Thus, the maximum value of the distance metric is $1 - 0 = 1$.

1.2.8. # Convert Full Distance Matrix to a Condensed Form

```
condensed_distance_matrix = squareform(
    distance_matrix, force="tovector", checks=False)
```

- Hierarchical clustering algorithms like *linkage* require a condensed distance matrix, which is a more compact representation. This is created using `squareform(distance_matrix)`.
- Main reason for it is computational efficiency.

1.2.9. # Perform Hierarchical Clustering

```
print("Performing hierarchical clustering...")
linkage_matrix = linkage(condensed_distance_matrix, method="average")
```

- The *linkage* function performs hierarchical clustering using the “average” linkage method. It builds a tree structure (dendrogram) where each leaf corresponds to a name and clusters are formed based on average distances.
- This takes some time (~20 minutes) when done for large datasets.

1.2.10. # Generate Cluster IDs

```
threshold = 0.2
cluster_ids = fcluster(linkage_matrix, t=threshold, criterion="distance")
```

- The *fcluster* function cuts the dendrogram at a threshold distance (here, $t = 0.2$) to form clusters. Names within the same cluster are assigned the same *cluster_id*, as their semantic embeddings are close enough.
- The value of the threshold (between 0 [strict] and 1 [lenient]) decides how strict the model clusters. The stricter the model the higher the number of **False Negatives** and the more lenient, the higher the number of **False Positives**. (See the Terminology section at the end of this document for definitions.)
- Trying different threshold can be useful, because a larger dataset might require stricter rules.
- General Rule (based on five datasets): Larger datasets require stricter thresholds, as the likelihood of similar names belonging to different individuals increases => this leads to a higher probability of “random collisions” and thus for **False Positives**, which can be avoided by a lower (thus stricter) threshold:
 - $n < 10,000$: $t = 0.2$
 - $n > 10,000$: $t = 0.1$
- Nevertheless, it also depends on the language and the names in the dataset, as can be seen in the Spanish dataset, where many special characters and symbols are included, which makes it a special case in general. You may need to experiment with different thresholds. Aim to justify why the threshold you chose is optimal, considering the trade-off between **False Positives** and **False Negatives**. For more details on this trade-off, refer to point 2.
- To determine the best threshold, start with a more lenient value and gradually move towards stricter thresholds. Compare the performance of each threshold using the method in R described in point 2.

1.2.11. # Assign cluster IDs to the Data Frame

```
country_df["Panel_ID"] = cluster_ids
```

- The *cluster_ids* are added to the original Data Frame in a new column named *Panel_ID*. Each unique cluster ID represents a unique panel, grouping names that likely refer to the same “candidate” over time.

1.2.12. # Save the Results

```
Output_path = os.path.join(os.path.expanduser(
    "~"), "Downloads", "chile_PanelID.csv")
country_df.to_csv(output_path, index=False)
print(f"Results saved to: {output_path}")
```

- The modified DataFrame is saved to a CSV file (*chile_PanelID.csv*) in the *Downloads* directory.

2. Test performance of Large Language Model in R

⇒ **Link:** https://github.com/oettlquin/Politikernamen-Panel-ID/blob/main/One-Page_Script.R

- **Load Dataset in your R environment**

- `chile_PanelID <- read_csv("Downloads/chile_PanelID.csv")`

- **Filter rows which have more than 1 Panel ID**

- `df_filtered <- chile_PanelID %>% group_by(Panel_ID) %>% filter(n() > 1) %>% ungroup()`

- `View(df_filtered)`

- Gives you an overview of those candidates which appear more than once in the Dataset.

- **Filter candidate names, which are not exactly the same, but have the same Panel ID**

- `df_different_names <- df_filtered %>% group_by(Panel_ID) %>% filter(n_distinct(candidate) > 1) %>% ungroup()`

- `View(df_different_names)`

- Shows you those candidates which have at least one difference in their names with at least one of their “Panel Partners”; helps you to find **False Positives** more easily.

- As mentioned earlier, the smaller the threshold, the stricter the model becomes. This reduces the number of **False Positives** but may increase the number of **False Negatives**, which are usually harder to detect. Therefore, there is a trade-off between the effort required to filter out **False Positives** and the risk of generating too many **False Negatives**.

- **Compare different thresholds**

- `chile_PanelID_t2 <- read_csv("Downloads/chile_PanelID_t2.csv")`

- `chile_PanelID_t1 <- read_csv("Downloads/chile_PanelID_t1.csv")`

- Load the datasets with different thresholds (here: 0.1 and 0.2), which you downloaded before from VSCode.

- `# Filter rows which have more than 1 PanelID`

- `df_filtered_t2 <- chile_PanelID_t2 %>%`

- `group_by(Panel_ID) %>%`

- `filter(n() > 1) %>%`
- `ungroup()`
- `# Filter candidate names, which are not exactly the same, but have the same PanelID`
- `df_different_names_t2 <- df_filtered_t2 %>%`
- `group_by(Panel_ID) %>%`
- `filter(n_distinct(candidate) > 1) %>%`
- `ungroup()`
- What we previously did for a single dataset, we now extend to two datasets to enable comparison.
- `# Filter rows which have more than 1 PanelID`
- `df_filtered_t1 <- chile_PanelID_t1 %>%`
- `group_by(Panel_ID) %>%`
- `filter(n() > 1) %>%`
- `ungroup()`
- `# Filter candidate names, which are not exactly the same, but have the same PanelID`
- `df_different_names_t1 <- df_filtered_t1 %>%`
- `group_by(Panel_ID) %>%`
- `filter(n_distinct(candidate) > 1) %>%`
- `ungroup()`
- `library(dplyr)`
- `# Find candidates in t1 but not in t2`
- `candidates_only_in_t1 <- df_different_names_t1 %>%`
- `filter(!candidate %in% df_different_names_t2$candidate)`
- `# Find candidates in t2 but not in t1`
- `candidates_only_in_t2 <- df_different_names_t2 %>%`
- `filter(!candidate %in% df_different_names_t1$candidate)`
- Create two tibbles to highlight the differences between the two thresholds, specifically identifying candidate names that exist in one dataset but not the other. According to the logic of hierarchical clustering, the smaller (stricter)

threshold should never include candidates that are absent from the larger (more lenient) threshold.

- # Display results
- print("Candidates in t1 but not in t2:")
- print(candidates_only_in_t1)
- print("Candidates in t2 but not in t1:")
- print(candidates_only_in_t2)

3. Create an Excel File and Save

- library(openxlsx)
- write.xlsx(chile_PanelID, "/your_storage_location/chile_panelID.xlsx", rowNames = FALSE)
- Save it to your storage location.

Explanation of Terminology:

- **False Positives**: Candidates that have the same Panel ID, but are not the same person.
- **False Negatives**: Candidates that have a different Panel ID, but are the same person.